

LAB MANUAL

CSE 0613-1202: Object Oriented Programming Lab

Lab Session: Classes, Attributes, Methods & Function Types

Field	Details
Course Code	CSE 0613-1202
Course Title	Object Oriented Programming Lab
Lab Topic	Introduction to Classes, Objects, Attributes & Methods
Language Used	Java
Session Duration	3 Hours (Demonstration + Group Task + Home Task)
Prerequisite	Basic programming concepts (variables, data types, operators)

1. Lab Objectives

By the end of this lab session, students will be able to:

- Explain the difference between a class and an object.
- Identify and declare attributes (instance variables) inside a class.
- Define methods to represent the behavior of a class.
- Understand what actually happens in memory when an object is created.
- Differentiate between constructors, accessor/mutator methods, static methods, and overloaded methods.
- Work in a group to design a complete class from a real-world scenario.

2. Theory: Classes, Objects, Attributes and Methods

2.1 What is a Class?

A class is a blueprint or template. It does not occupy memory for data by itself — it only describes what an object made from it will look like and how it will behave. Think of a class as an architectural drawing of a house: the drawing itself is not a house you can live in, but every house built from that drawing will have the rooms and features described in it.

2.2 What is an Object?

An object is a real instance created from a class. When an object is created using the 'new' keyword, memory is actually allocated on the heap to store that object's own copy of the attributes described in the class.

Behind the scenes: When you write 'new Student()', the JVM (Java Virtual Machine) reads the Student class blueprint, allocates a block of memory on the heap large enough to hold one copy of every attribute, sets numeric attributes to 0 and object references to null by default, runs the constructor to set proper initial values, and finally returns the memory address of this block. This address is what gets stored in the reference variable on the left-hand side. This is why two objects of the same class, even with identical attribute values, are still two separate memory blocks.

2.3 What are Attributes?

Attributes (also called instance variables or fields) are the data or characteristics that describe an object. They are declared inside the class body but outside any method. Every object created from the class gets its own separate copy of these attributes in memory.

2.4 What are Methods?

Methods are blocks of code declared inside a class that define the behavior or actions an object can perform. Unlike attributes, the method's code itself is stored only once in memory (not duplicated per object) — every object of that class shares and executes the same method code, but each call works on that particular object's own attribute values.

3. Example 1: A Class with Attributes Only

Let us start with the simplest possible class — one that only stores data, with no behavior yet.

```
public class Student {  
    // Attributes (instance variables)  
    String name;  
    int age;  
    double cgpa;  
}
```

Line-by-line explanation

Code	Explanation
<code>public class Student {</code>	Declares a new class named Student. 'public' means this class can be accessed from other classes/packages.
<code>String name;</code>	An attribute of type String. Every Student object will have its own 'name' box in memory.
<code>int age;</code>	An attribute of type int, storing a whole number for the student's age.
<code>double cgpa;</code>	An attribute of type double, storing a decimal number for CGPA.
<code>}</code>	Closes the class body.

Creating and using objects of this class

```
public class Main {
    public static void main(String[] args) {

        Student s1 = new Student(); // Object 1 created
        s1.name = "Rafiq";
        s1.age = 20;
        s1.cgpa = 3.75;

        Student s2 = new Student(); // Object 2 created
        s2.name = "Nadia";
        s2.age = 21;
        s2.cgpa = 3.90;

        System.out.println(s1.name + " - " + s1.cgpa);
        System.out.println(s2.name + " - " + s2.cgpa);
    }
}
```

Behind the scenes: s1 and s2 are two completely separate blocks of memory on the heap. Changing s1.name never affects s2.name, because each object carries its own copy of every attribute declared in the class. The variables s1 and s2 themselves live on the stack and only hold the heap address of their respective objects — they are references, not the objects themselves.

4. Example 2: Adding Methods (Behavior) to a Class

A class becomes truly useful once it can act on its own data. Let's rebuild the same idea with a Car class that has both attributes and methods.

```
public class Car {

    // Attributes
    String brand;
    int speed;
    boolean isEngineOn;
```

```

// Method 1: change the object's state
void startEngine() {
    isEngineOn = true;
    System.out.println(brand + " engine started.");
}

// Method 2: change the object's state using a parameter
void accelerate(int increaseBy) {
    if (isEngineOn) {
        speed = speed + increaseBy;
        System.out.println(brand + " is now at " + speed + " km/h.");
    } else {
        System.out.println("Start the engine first!");
    }
}

// Method 3: report the object's current state
void displayInfo() {
    System.out.println("Brand: " + brand + ", Speed: " + speed
        + ", Engine On: " + isEngineOn);
}
}

```

What is happening 'behind the scenes'

- brand, speed, isEngineOn are attributes — each Car object stores its own copies.
- startEngine(), accelerate(int), displayInfo() are methods — their compiled instructions live once in memory, shared by every Car object.
- When you call myCar.accelerate(10), Java secretly passes a hidden reference to myCar itself (called 'this') into the method, so the method knows exactly whose speed and isEngineOn to read and update.
- Because accelerate() checks isEngineOn before changing speed, the object protects its own data — this controlled access is the core idea of encapsulation.

```

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.brand = "Toyota";
        myCar.speed = 0;

        myCar.startEngine();    // turns engine on
        myCar.accelerate(20);    // speed becomes 20
        myCar.accelerate(15);    // speed becomes 35
        myCar.displayInfo();
    }
}

```

5. Attributes vs Methods — Quick Comparison

Aspect	Attribute	Method
Represents	State / data of an object	Behavior / action of an object
Declared as	variable inside class body	function inside class body
Memory	One copy per object (heap)	One copy shared by all objects

Aspect	Attribute	Method
Example	String name;	void displayInfo() { ... }
Accessed using	object.attribute	object.method(arguments)

6. Types of Methods (Functions) in a Class

Today we will also practice different kinds of methods that appear inside a class. Each type serves a different purpose.

6.1 Constructors

A constructor is a special method with the same name as the class and no return type. It runs automatically the moment an object is created with 'new', and its job is to give the attributes their correct starting values.

```
public class Book {
    String title;
    double price;

    // Default constructor
    Book() {
        title = "Untitled";
        price = 0.0;
    }

    // Parameterized constructor
    Book(String t, double p) {
        title = t;
        price = p;
    }
}
```

Behind the scenes: If you write no constructor at all, Java silently inserts an invisible empty default constructor for you at compile time. The moment you write ANY constructor yourself, Java stops adding that invisible one — this is why 'new Book()' stops compiling if you only defined Book(String, double).

6.2 Accessor (Getter) and Mutator (Setter) Methods

Attributes are often marked 'private' so outside code cannot touch them directly. Getters and setters are public methods that give safe, controlled read/write access.

```
public class Book {
    private double price;

    double getPrice() {           // accessor / getter
        return price;
    }

    void setPrice(double p) {     // mutator / setter
        if (p >= 0) {
            price = p;
        } else {
            System.out.println("Price cannot be negative.");
        }
    }
}
```

6.3 Void Methods vs Value-Returning Methods

```
void printMessage() {           // void: performs an action, returns nothing
    System.out.println("Hello!");
}

int square(int n) {             // returns a value of type int
    return n * n;
}
```

6.4 Instance Methods vs Static Methods

An instance method belongs to a specific object and can use that object's attributes (e.g., `myCar.accelerate(10)`). A static method belongs to the class itself, is called without any object (e.g., `MathHelper.square(5)`), and cannot directly touch instance attributes because it does not act on any particular object.

```
public class MathHelper {
    static int square(int n) {    // static: called as MathHelper.square(5)
        return n * n;
    }
}
```

6.5 Method Overloading

Overloading means writing several methods with the same name in the same class, but with a different number or type of parameters. Java decides which one to run based on the arguments used in the call.

```
int add(int a, int b) {
    return a + b;
}

double add(double a, double b) {
    return a + b;
}

int add(int a, int b, int c) {
    return a + b + c;
}
```

7. In-Lab Group Task

Form groups of 3–4 students. Each group will pick ONE scenario below and implement it completely in Java, including a Main class that creates at least two objects and demonstrates every method.

Scenario A — BankAccount

- Attributes: accountNumber, accountHolderName, balance
- Constructor: parameterized constructor to set accountNumber and accountHolderName, balance starts at 0
- Methods: deposit(double amount), withdraw(double amount) — must reject withdrawal if amount > balance, checkBalance() that returns the balance, displayAccountInfo()

Scenario B — LibraryBook

- Attributes: title, author, isIssued (boolean)
- Constructor: parameterized constructor to set title and author, isIssued starts as false
- Methods: issueBook() — sets isIssued to true only if it is currently false, returnBook() — sets isIssued to false, displayStatus()

Scenario C — Employee

- Attributes: empId, name, basicSalary
- Constructor: parameterized constructor for all three attributes
- Methods: calculateBonus() that returns 10% of basicSalary, calculateGrossSalary() that returns basicSalary + bonus, displayPayslip()

Group Task Requirements

1. Use a private access modifier for all attributes.
2. Provide getter and setter methods for every attribute.
3. Include at least one static method (e.g., a counter of how many objects were created).
4. Create at least 2 objects in Main and print their details.
5. Be ready to explain, in your own words, what happens in memory when your objects are created.

8. Home Task (Individual)

Complete the following individually and submit before the next lab session:

1. Design a class named Rectangle with attributes length and width.
2. Add a parameterized constructor to initialize both attributes.
3. Add a method calculateArea() that returns length * width.
4. Add a method calculatePerimeter() that returns 2 * (length + width).
5. Overload a method scale() twice: once taking a single int factor that scales both dimensions equally, and once taking two separate int factors for length and width.
6. Add a static method that keeps count of how many Rectangle objects have been created so far.
7. In Main, create 3 Rectangle objects with different dimensions, call every method, and print all results neatly.

Submission: Bring a hardcopy of written code and your source code (.cpp file) with a screenshot of the working output to the next lab session.